

컴퓨터비전_인공지능(AI)개론

9. 다중 분류1

- 문제 정의하기
- 여러 개의 분류기
- 가중치 행렬
- 소프트맥스 함수
- 교차 엔트로피 함수
- 다중 분류 모델에서 예측 함수와 손실 함수의 관계
- 데이터 준비
- 모델 정의
- 경사 하강법
- 결과 확인
- 입력 변수의 4차원화

▶ 다중 분류: 여러 선택지 중 하나를 고르는 기술

이진 분류가 동전의 앞/뒤를 맞추는 게임이었다면,

다중 분류는 여러 개의 문 중에 진짜 보물이 숨겨진 단 하나의 문을 찾는 게임과 같습니다.

선택지가 많아진 만큼, 더 정교한 전략이 필요합니다.

붓꽃의 품종을 'setosa', 'versicolor', 'virginica' 세 가지로 분류하려면,

모델은 "이 붓꽃이 setosa일 확률은 30%, versicolor일 확률은 60%, virginica일 확률은 10%입니다"와 같이

각 항목에 대한 확률을 모두 제시해야 합니다.

이진 분류

모델의 출력이 '정답일 확률' 단 하나

1차원 출력

다중 분류

각 클래스별 확률을 모두 계산

N차원 출력

가장 큰 변화는 출력의 차원입니다.

이진 분류 모델의 최종 출력은 1차원이었지만, N개의 그룹으로 분류하는 다중 분류 모델의 출력은 N차원이 됩니다.

01

분류기 0

"이 데이터가 0번 그룹에 속할 확률은?"

02

분류기 1

"이 데이터가 1번 그룹에 속할 확률은?"

03

분류기 2

"이 데이터가 2번 그룹에 속할 확률은?"

모델 내부에 N개의 작은 분류기가 있고, 각 분류기가 특정 그룹을 전담하는 것과 같습니다.

모델은 입력 데이터에 대해 모든 분류기의 확률 값을 계산하고,

이 중 가장 높은 확률을 제시하는 분류기의 그룹을 최종 예측값으로 선택합니다.

이진 분류

가중치 벡터 사용

하나의 레시피

입력 벡터와 가중치 벡터의 내적으로 하나의 출력 계산

다중 분류

가중치 행렬 사용

여러 레시피가 담긴 요리책

입력 벡터와 가중치 행렬의 곱셈으로 여러 개의 출력 동시 계산

출력이 1개에서 N개로 늘어남에 따라, 모델의 파라미터(가중치)도 형태가 바뀝니다.

동일한 재료(입력)를 가지고 각기 다른 레시피(가중치 행)를 적용하여 여러 맛의 요리(출력)를 각각 계산하는 셈입니다.

이진 분류에서 시그모이드 함수가 출력을 0과 1 사이의 확률로 바꿔주었다면, 다중 분류에서는 소프트맥스 함수가 그 역할을 합니다.



이름이 '소프트맥스'인 이유는 가장 큰 값만 1로 만드는 극단적인 방식(hard max)이 아니라, 가장 큰 값의 확률이 가장 크도록 하면서도 다른 값들에게도 어느 정도의 확률을 부드럽게(soft) 할당해주기 때문입니다.

다중 분류에서도 교차 엔트로피를 사용하지만, 계산 방식에 차이가 있습니다.

로그 적용

소프트맥스 함수의 출력(각 클래스일 확률)에 로그를 취합니다.

정답 요소 추출

여러 로그 값 중에서 실제 정답에 해당하는 클래스의 값만 추출합니다.

손실 계산

이 값에 음수를 붙여 손실로 사용합니다.
확률이 1에 가까울수록 손실이 작아집니다.

이론적으로는 선형 함수 → 소프트맥스 함수 → 로그 함수 → 정답 요소 추출의 흐름을 따르지만,
파이토치는 더 효율적인 방법을 제공합니다.

`nn.CrossEntropyLoss`는 내부적으로 소프트맥스 + 로그 + 정답 요소 추출 과정을 모두 포함하고 있어 수치적으로 안정적이고 효율적입니다.

예측 모델 (net)

활성화 함수 없이 `nn.Linear`의 출력(raw score, logit)을 그대로
반환

손실 함수

`nn.CrossEntropyLoss`가 모델의 출력을 받아
내부적으로 소프트맥스를 적용하고 손실을 계산



데이터 준비

150개의 붓꽃 데이터를 훈련용 75개, 검증용 75개로 분할합니다.

시각화를 위해 꽃받침 길이와 꽃잎 길이 2개 특징을 선택합니다.



모델 정의

2개 입력, 3개 출력을 가진 선형 모델을 정의합니다.

가중치는 벡터가 아닌 행렬 형태가 됩니다.



학습 및 평가

경사 하강법을 통해 모델을 학습시키고, 손실과 정확도 변화를 관찰합니다.

모델을 학습시키기 전, 데이터를 모델이 이해할 수 있는 형태로 준비해야 합니다.

데이터 분할

과적합 방지를 위해 훈련 데이터와 검증 데이터로 분할합니다.

모든 데이터로 학습하면 모델이 데이터를 단순히 '외워버릴' 수 있습니다.

데이터 시각화

산점도를 통해 데이터의 분포를 확인합니다.

선형 모델이 데이터를 잘 분류할 수 있을지 미리 가늠해볼 수 있습니다.

텐서 변환

넘파이 배열을 파이토치 텐서로 변환합니다.

입력 데이터는 float, 정답 레이블은 반드시 long(정수) 타입으로 지정해야 합니다.

모델은 (배치 크기, 클래스 수) 형태의 텐서를 출력합니다.

75개 데이터에 대해 3개 클래스를 예측하면 (75, 3) 텐서가 나옵니다.

torch.max(outputs, 1) 사용법

outputs 텐서에 대해 두 번째 차원(dim=1, 행별로)을 기준으로 최댓값을 찾습니다.

반환값:

- values: 실제 최댓값들이 담긴 텐서
- indices: 최댓값이 위치한 인덱스들이 담긴 텐서

최종 예측

우리가 필요한 것은 `indices`입니다.

이 인덱스가 바로 모델이 예측한 클래스 레이블이 됩니다.

```
torch.max(outputs, 1)[1]
```

학습 곡선은 에포크가 진행됨에 따라 손실과 정확도가 어떻게 변하는지를 보여주는 중요한 지표입니다.

손실 곡선

이상적인 손실 곡선은 반복 횟수가 늘어남에 따라 부드럽게 감소하여 낮은 값으로 수렴합니다.

안정성 확인

훈련 손실과 검증 손실이 비슷하게 움직이면 학습이 안정적으로 이루어지고 있음을 의미합니다.

1

2

3

정확도 곡선

손실과 반대로, 반복 횟수에 따라 부드럽게 증가하여 높은 값으로 수렴합니다.

nn.CrossEntropyLoss를 사용했기 때문에, 모델은 소프트맥스를 거치지 않은 원시 출력값(logits)을 반환합니다.

원시 출력 (Logits)

모델이 직접 출력하는 값

확률이 아니므로 해석이 어려움

예: [2.1, 0.8, -1.3]

소프트맥스 적용 후

0과 1 사이의 확률값으로 변환

모든 값의 합이 1

예: [0.65, 0.30, 0.05]

모델의 실제 예측 성능을 체감하고 싶을 때는 출력값에 직접 소프트맥스 함수를 적용해야 합니다.

이를 통해 모델이 각 클래스에 대해 얼마나 확신하는지를 명확하게 확인할 수 있습니다.

NLLLoss는 Negative Log Likelihood Loss의 약자로, 교차 엔트로피 손실 계산의 핵심적인 부분을 담당합니다.

NLLLoss의 역할: 정답 요소 추출

교차 엔트로피 계산의 마지막 단계에서 정답 클래스에 해당하는 예측값만을 선택하여 손실을 계산합니다.

- 모델의 원시 출력(Logits)에 소프트맥스를 적용해 확률을 구한 후 로그를 취합니다.

NLLLoss는 이 로그 확률들 중에서 실제 정답 레이블이 가리키는 위치의 값만 뽑아 음수(-)를 붙여 손실로 사용합니다.

예시: 정답 레이블 `[0, 1, 2]`에 대해 모델이
`[log\_prob\_0, log\_prob\_1, log\_prob\_2]`를 반환했을 때,
NLLLoss는 $-(\log_prob_0 + \log_prob_1 + \log_prob_2) / 3$ 을 계산합니다.

파이토치의 효율적인 설계

다른 딥러닝 프레임워크와 달리,
파이토치는 정답 레이블에 대한 원-핫 인코딩 전처리 과정이
필요 없습니다.

- 파이토치는 정수형 레이블 `[0, 1, 2]`를 직접 받아 해당 인덱스에 해당하는 예측값을 추출하는 방식으로 동작하여, 불필요한 메모리 사용과 연산을 줄여줍니다.

표준 패턴(선형 모델 + CrossEntropyLoss) 외에도 다양한 구현 방법이 있습니다.

패턴 1 (표준)

선형 모델 + CrossEntropyLoss

가장 안정적이고 권장되는 방식

패턴 2

LogSoftmax + NLLLoss

표준 패턴과 동일한 결과

패턴 3

Softmax + 수동 log + NLLLoss

번거롭고 수치적으로 불안정

패턴 2는 모델이 LogSoftmax까지 계산하여 로그 확률을 출력하고,

손실 함수는 NLLLoss를 사용하여 마지막 '추출' 작업만 담당합니다.

패턴 3은 권장되지 않는 방식입니다.

1 출력 차원의 확장

1개 출력에서 N개 출력으로,
가중치 벡터에서 가중치 행렬로 변화

2 소프트맥스의 역할

N개의 출력값을 확률 분포로 변환, 모든 확률의 합이 1

3 파이토치의 효율적 설계

CrossEntropyLoss가 소프트맥스부터
손실 계산까지 한 번에 처리

4 더 많은 정보, 더 나은 성능

입력 변수를 늘리면 정확도는 동일하더라도
확신도(낮은 손실)가 향상

다중 분류를 이해하면 숫자 이미지 분류, 뉴스 기사 카테고리 분류 등 훨씬 다채롭고 현실적인 문제들을 해결할 수 있습니다.